Cross-ISA: a laptop predicting real x86 production hardware

Generated by `scripts/analysis/laptop_to_ec2_evaluation.py`. Raw numbers: `results/laptop_to_ec2_metrics.json`, pairs: `results/laptop_to_ec2_pairs.csv`.

This is the only experiment in the project that uses real production machines on a different instruction set. dev = Apple M1 (arm64, PostgreSQL 15.17). prod = the five EC2 instances the project was built on: c5a.xlarge and m5a.xlarge (4 vCPU), r5n.large, z1d.large, c7i.large (2 vCPU) — all **x86_64**. 19 queries × 5 machines = **95 pairs**.

Why the operator model is not evaluated here

The five EC2 instances are terminated. `operator_probe.py` executes SQL on the machine, so no operator rates and no parallel-speedup measurement exist for them, and none can be obtained. Deriving rates from the runtimes being predicted would be circular. What this evaluation tests is therefore the component that transfers without a new probe — **few-shot calibration** — on top of the best scalar signature that exists on both sides.

Zero-shot

predictor	MAPE %	MdAPE %	q-error p50	q-error p95
naive (assume prod = dev)	129.39	57.44	1.636	6.297
`cmp_only`, compute probed at `nproc` (8 threads)	337.75	203.25	3.033	10.918
`cmp_only`, probed at achievable DOP (3 threads)	160.68	74.73	1.867	6.291

These reproduce the previously published 324% / 153% to within the exclusion of two approximate queries, which confirms the pipeline against the prior result.

Few-shot calibration

Calibrating on k queries measured on production, scored **only on the queries not revealed**:

k	random	naive (same held-out subset)
0	160.68	129.39
1	80.04	132.15
2	84.35	119.50
3	98.37	124.99
5	**55.88**	93.29
8	109.92	139.27

`spread` **selection is inapplicable here and the script now says so rather than producing a number.** `cmp_only` depends only on the machine, so every query carries the *same* predicted scaling and a rule that spreads over predicted scaling has no range to spread over — it silently degenerated into picking queries in name order. (In the operator model predicted scaling varies per query, because it depends on the plan, which is why `spread` is meaningful there and not here. That is a real precondition for the rule, not a detail.)

Calibration absorbs the entire `nproc`-vs-DOP bug. The two `cmp_only` variants differ by a constant factor (61.82/35.62 = 1.736), and a per-machine multiplicative correction removes any constant exactly — so their post-calibration curves are **identical**. A definition bug that cost 337.75% against 160.68% zero-shot becomes *completely irrelevant* after one calibration query. Worth stating plainly in the paper: calibration does not merely improve a scalar predictor, it makes the choice of scalar unidentifiable.

The decisive result: a scalar model cannot work cross-ISA

Every predictor in this project's lineage — bandwidth ratio, compute ratio, the analytical roofline, and any of them plus a calibration — multiplies the dev runtime by **one number per machine pair**. Such a model cannot express variation in the scaling factor *across queries* on the same machine. Dividing out each machine's median scaling isolates exactly that irreducible residual:

	cross-ISA (laptop → EC2 x86)	**same-ISA (laptop → containers)**
residual spread after a *perfect* per-machine scalar	**44.5× max/min**	13.6×
MAPE floor of any per-machine-scalar predictor	**59.7%**	**22.2%**
best measured	55.88% (k=5)	28.54% (zero-shot)

Per-machine scaling spread across queries: c5a 19×, m5a 18×, c7i 26×, r5n 38×, z1d 43×.

Two conclusions, and they are the paper.

1. Cross-ISA, the measured result is already at the floor. 55.88% against a floor of 59.7% — a per-machine scalar predictor has nothing left to give. The original project's failure was not a tuning problem, a sample-size problem or a probe-calibration problem: it was **the wrong model class for the problem it was attempting.** Crossing arm64 → x86_64 changes the *relative* cost of operators, and one number per machine cannot represent that.

2. Same-ISA, the current model is near-optimal for its class. 28.54% against a floor of 22.2%. There is little left to win by calibrating better; the remaining headroom needs per-query capability information.

Together these say exactly what to build next: **per-operator rates measured on an x86 machine.** That is the one experiment that could beat the 59.7% floor, because per-operator capability is per-query information by construction — a query that is 80% sort and a query that is 80% sequential scan get different scaling factors, which no scalar can do.

Caveats

- **EC2's `max_parallel_workers_per_gather = 2` is an inference**, not a re-measurement: it is PostgreSQL's default, the collection guide never mentions tuning it, and the instances were terminated before it could be checked. The M1 side is measured at 2.
- **q15 excluded** (no EC2 counterpart). **q16 and q20 excluded** from the primary result as approximate laptop reconstructions (`results/query_reconstruction_validation.csv`); with them the set is 105 pairs.
- 19 queries, 5 machines, one scale factor, one engine version, single client.
- The compute scalar is FP-vector GFLOPS, which this project has already shown is the wrong quantity for database work. It is used here because it is the only signature that exists on both sides.
- k=8 scoring worse than k=5 is small-sample noise: only 11 queries remain held out, and the naive baseline on that subset also moves (93.29 → 139.27).